Section 4.4: Famous CNN Architectures

From LeNet-5 to ResNet, and Transfer Learning

AI-IM Class — Lecture Notes

With convolution, pooling and the classifier head (Sections 4.2 and 4.3) we can build any CNN. The question is *which* CNN to build: how many layers, what kernel sizes, how many channels. The answers came from a short series of famous architectures, each introducing one idea that is still used today. In this section we

- trace LeNet-5, the first successful CNN, layer by layer,
- follow the ImageNet competition that made CNNs famous,
- study the key idea of AlexNet, VGG, GoogLeNet and ResNet, with a hand calculation for each, and
- reuse a pretrained network for our own problem (*transfer learning*).

1 LeNet-5 (1998)

LeNet-5, by Yann LeCun and colleagues, read handwritten digits on bank cheques. It already has the structure of Section 4.3: two conv–pool blocks followed by fully connected layers.

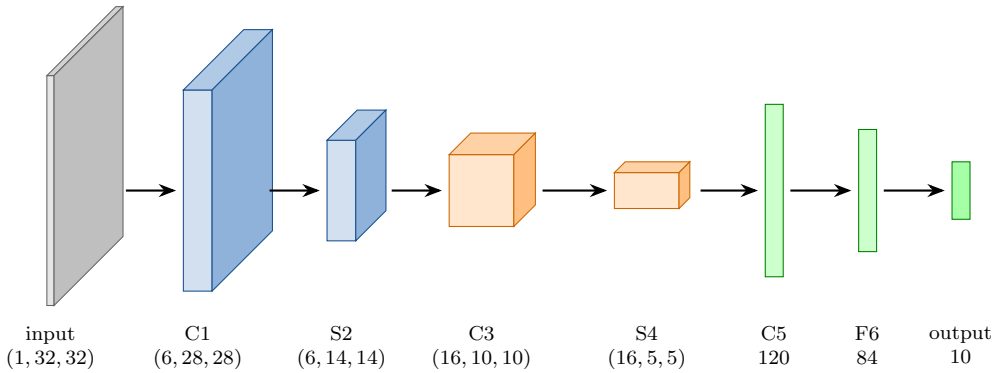


Figure 1: LeNet-5. C = convolution (5×5, valid), S = subsampling (2×2 average pooling), F = fully connected.

The input is a 32×32 grayscale image (MNIST digits padded from 28×28). All convolutions use 5×5 kernels with stride 1 and no padding; all pooling layers are 2×2 with stride 2. We use the common modern version: ReLU activations, average pooling, every C3 filter connected to all 6 input channels, and a linear output layer.

Exercise 1 Trace LeNet-5

Layer	Operation	Output shape	Parameters
Input		$(1, 32, 32)$	—
C1	6 filters 5×5	$(6, 28, 28)$	$(1 \cdot 25 + 1) \cdot 6 = 156$
S2	avg pool 2×2	$(6, 14, 14)$	0
C3	16 filters 5×5	$(16, 10, 10)$	$(6 \cdot 25 + 1) \cdot 16 = 2\,416$
S4	avg pool 2×2	$(16, 5, 5)$	0
C5	120 filters 5×5	$(120, 1, 1)$	$(16 \cdot 25 + 1) \cdot 120 = 48\,120$
F6	linear	(84)	$(120 + 1) \cdot 84 = 10\,164$
Output	linear	(10)	$(84 + 1) \cdot 10 = 850$
Total			61 706

Why is C5 called a convolution although it behaves like a fully connected layer?

Its 5×5 kernel exactly covers the 5×5 input, so there is only one position and each filter sees every input value.

The original LeNet-5 used scaled tanh activations, pooling layers with one trainable weight and bias per channel, and a hand-designed table that connected each C3 filter to only some of the S2 channels (1 516 parameters instead of 2 416), to save computation in 1998. Its total is still about 60 thousand parameters.

2 The ImageNet Challenge

LeNet-5 worked on small grayscale digits. Natural photos are much harder: thousands of object types, varying lighting, viewpoint and background. The **ImageNet Large Scale Visual Recognition Challenge (ILSVRC)** ran from 2010 to 2017. Its classification task had about 1.2 million training images in 1 000 classes. Performance is measured by the **top-5 error**: a prediction counts as correct if the true class is among the model's five most probable classes.

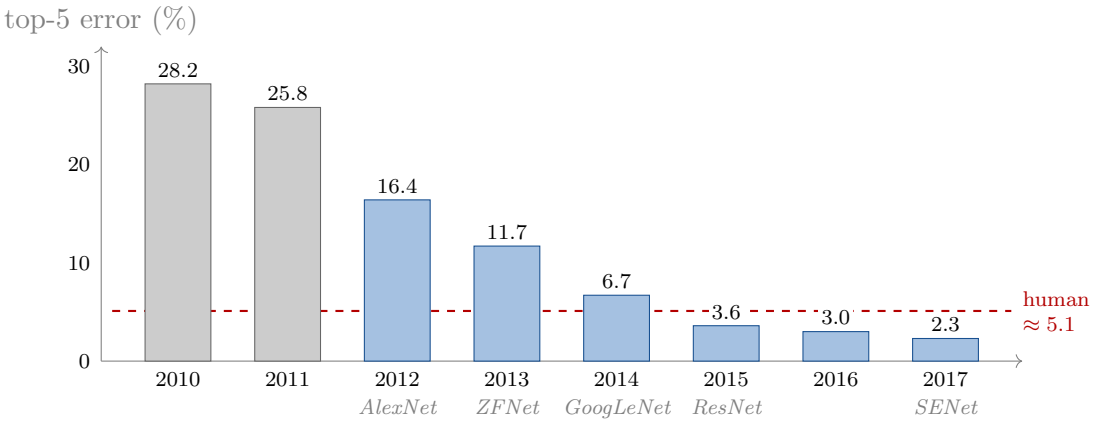


Figure 2: Winning top-5 error in the ILSVRC classification task. Grey: methods based on hand-designed features; blue: CNNs. The human estimate is Andrej Karpathy's result on a test sample.

The jump in 2012 is the moment deep learning took over computer vision. By 2015 CNNs beat the human estimate. Each of the architectures below won (or nearly won) one of these years.

3 AlexNet (2012): Scale It Up

AlexNet (Krizhevsky, Sutskever and Hinton) is LeNet made bigger: 5 convolutional and 3 fully connected layers, about 60 million parameters, trained on 224×224 colour images. Its architecture was not new; what made it work were practical choices that are still standard today:

- **ReLU** instead of tanh, which trained several times faster (Section 3.4);
- **GPUs**: two gaming graphics cards, for about a week;
- **data augmentation**: random crops and flips of the training images, to get more data for free;
- **dropout** in the fully connected layers, against overfitting (we return to it in a later topic).

Exercise 2 AlexNet's first layer

The first layer has 96 filters of size 11×11 with stride 4 and no padding. In the original implementation the input is $(3, 227, 227)$.

(a) Output size: $\lfloor (227 - 11)/4 \rfloor + 1 = 55$, so the output shape is $(96, 55, 55)$.

(b) Parameters: $(3 \cdot 121 + 1) \cdot 96 = 34\,944$.

4 VGG (2014): Only Small Kernels

VGG (Simonyan and Zisserman, Oxford) asked how far a very simple design can go. It uses only one kind of convolution, 3×3 with stride 1 and same padding, and only 2×2 max pooling, repeated in blocks until the network has 16 or 19 layers. Each block halves the feature maps and doubles the channels: $64 \to 128 \to 256 \to 512$.

Why only 3×3? Two stacked 3×3 layers have the same receptive field as one 5×5 layer (Section 4.1), but they are cheaper and have one more ReLU.

Exercise 3 Two small kernels vs. one large

All layers have C input and C output channels. Ignore biases.

Option	Receptive field	Weights	Weights for $C = 64$
one 5×5 layer	5×5	$25C^2$	102 400
two 3×3 layers	5×5	$18C^2$	73 728
one 7×7 layer	7×7	$49C^2$	200 704
three 3×3 layers	7×7	$27C^2$	110 592

VGG-16 reached 7.3% top-5 error, but it has about 138 million parameters, most of them in its three fully connected layers. Its uniform design made it popular as a starting point for other tasks.

5 GoogLeNet / Inception (2014): Go Wide, Cheaply

GoogLeNet (Szegedy et al., Google) won 2014 with 6.7% top-5 error using about 7 million parameters, roughly 20 times fewer than VGG. Its building block is the **Inception module**: instead of choosing one kernel size, it applies several in parallel and concatenates the results along the channel dimension.

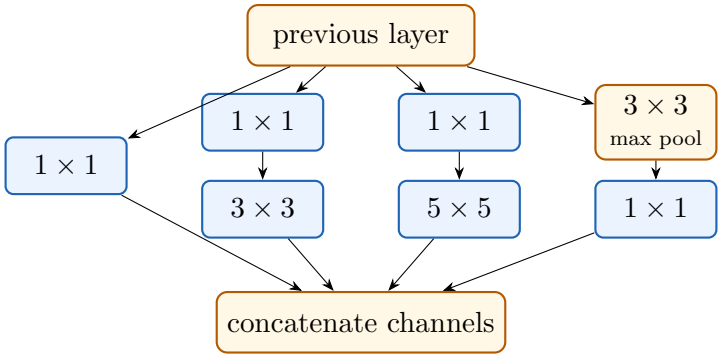


Figure 3: An Inception module. Four branches see the same input with different receptive fields. In the second and third branches a 1×1 convolution first shrinks the number of channels, so the expensive 3×3 and 5×5 convolutions become cheap.

The 1×1 convolution. A 1×1 kernel looks at a single pixel, but across *all* its channels. At every pixel it computes a weighted sum of the C_{in} channel values, which is exactly a small fully connected layer applied at each pixel separately. It changes the number of channels without touching the height and width.

Exercise 4 The 1×1 bottleneck

(a) One pixel has channel values $(2, -1, 3)$. A 1×1 layer has two filters with weights $(1, 0, 1)$ and $(0, 2, -1)$ and no bias. The pixel's output is $\big(5,\ -5\big)$.

(b) An input of shape $(256, 28, 28)$ goes through a 5×5 convolution with 32 filters (same padding). Ignoring biases, it has $256 \cdot 25 \cdot 32 = 204\,800$ weights.

(c) Insert a 1×1 convolution with 16 filters first. The weights are now

$$\underbrace{256 \cdot 16 = 4\,096}_{1\times1} + \underbrace{16 \cdot 25 \cdot 32 = 12\,800}_{5\times5} = 16\,896,$$

about 12 times fewer. The number of multiplications shrinks by the same factor.

6 ResNet (2015): Skip Connections

If deeper is better, why not stack 100 layers? He et al. (Microsoft Research) found that plain deep networks did not just overfit: a 56-layer network had a *higher training error* than a 20-layer one. The deeper network was harder to *optimise*, even though it could in principle copy the 20-layer network and set the extra layers to the identity.

Their fix was the **residual block**. Its layers compute a correction $F(\mathbf{H})$, which is *added* to the block's input through a **skip connection** (shortcut):

$$\text{output} = \text{ReLU}\big(F(\mathbf{H}) + \mathbf{H}\big).$$

If the best thing a block can do is nothing, it only needs $F = 0$, which is easy to learn (all weights near zero).

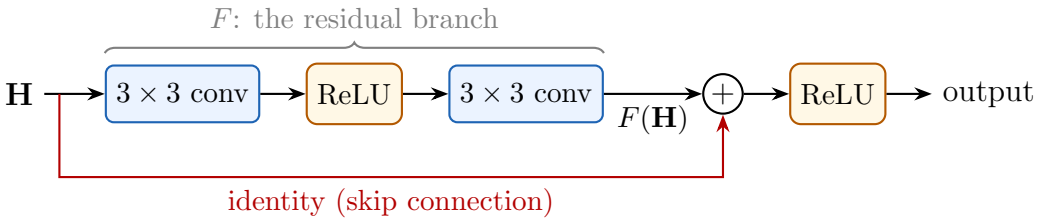


Figure 4: A residual block. The input skips past two convolutions and is added to their output. (Real ResNets also put a normalisation layer after each convolution, which we meet in a later topic.)

Why it helps the gradient. Recall the vanishing-gradient problem of Section 3.4: backpropagation multiplies one derivative per layer, and a product of many small numbers is tiny. Take one scalar "layer" $y = f(x)$ with derivative f'. The residual version $y = x + f(x)$ has derivative $1 + f'$. The 1 comes from the skip connection, and it does not shrink.

Exercise 5 Gradient through three layers

Each layer has $f' = 0.1$. Compute $\partial y / \partial x$ through three layers in a row.

$$\text{plain: } (f')^3 = 0.001, \qquad \text{residual: } (1 + f')^3 = 1.331.$$

Through 50 plain layers the gradient would be $0.1^{50} = 10^{-50}$.

With skip connections, He et al. trained networks with 152 layers and won 2015 with 3.6% top-5 error, better than the human estimate. When the two shapes differ (the block changes the number of channels or uses stride 2), the shortcut uses a 1×1 convolution to match them. Skip connections are now everywhere in deep learning, including the transformers behind large language models.

7 Comparison

Network	Year	Layers	Parameters	Top-5 error	Key idea
LeNet-5	1998	5	60 K	—	conv + pool + dense
AlexNet	2012	8	60 M	16.4%	ReLU, GPUs, augmentation
VGG-16	2014	16	138 M	7.3%	stacks of 3×3 kernels
GoogLeNet	2014	22	7 M	6.7%	Inception, 1×1 bottleneck
ResNet-152	2015	152	60 M	3.6%	skip connections

After 2015. Later designs mostly refine these ideas. *MobileNet* splits each convolution into a per-channel spatial step and a 1×1 step so that it runs on phones and embedded cameras. *EfficientNet* scales depth, width and image resolution together. *Vision Transformers* (2020) replace convolution with attention, and *ConvNeXt* (2022) showed that a modernised ResNet is just as good. CNNs remain the default choice when data or computing power is limited, as in most factory applications.

8 Transfer Learning

Training a ResNet from scratch needs a million labelled images and days of GPU time. A factory that wants to classify, say, three kinds of surface defect may have only a few hundred photos. **Transfer learning** solves this by reusing a network that was already trained on ImageNet.

The idea follows from Section 4.3. The feature extractor of an ImageNet network has learned general features (edges, textures, shapes) that are useful for almost any image. Only the classifier head is specific to the 1 000 ImageNet classes. So we

1. load the pretrained network,
2. replace its final linear layer with a new one that has one output per *our* class, and
3. train on our data: either only the new layer (*feature extraction*, with the rest frozen), or the whole network with a small learning rate (*fine-tuning*).

```python
import torchvision

model = torchvision.models.resnet18(weights="IMAGENET1K_V1")
for p in model.parameters():
    p.requires_grad = False              # freeze the feature extractor

model.fc = nn.Linear(512, 3)             # new head: 3 defect classes
optimizer = torch.optim.Adam(model.fc.parameters(), lr=1e-3)
```

Exercise 6 How much is trained?

ResNet-18 has about 11.7 million parameters. Its final global average pooling produces 512 features.

(a) The new head has $(512 + 1) \cdot 3 = 1\,539$ parameters.

(b) With the feature extractor frozen, we train about 0.013% of the network.

9 Summary

- LeNet-5 set the pattern: conv–pool blocks followed by dense layers. Tracing its shapes and parameters uses only the formulas of Sections 4.2 and 4.3.
- AlexNet (2012) showed that a large CNN trained on GPUs with ReLU, augmentation and dropout beats hand-designed features by a wide margin.
- VGG: stacks of 3×3 kernels give large receptive fields with fewer weights and more nonlinearity.
- GoogLeNet: parallel branches, with 1×1 convolutions to cut the number of channels cheaply.
- ResNet: skip connections, output $= \mathrm{ReLU}(F(\mathbf{H}) + \mathbf{H})$, keep gradients alive and make networks of 100+ layers trainable.
- Transfer learning reuses an ImageNet feature extractor and retrains only a new head, so a few hundred images can be enough.

Coming up. Section 4.5 goes beyond classifying a whole image: finding *where* objects are (detection, segmentation) and reading text (OCR).